PROGRAM 8: Implement a Context-Aware Chatbot Using LangChain, Retrieval and an LLM

AIM

To implement a context-aware chatbot that retrieves relevant information from a knowledge base and generates answers using previous conversation context.

ALGORITHM

1. Install the required libraries.
2. Load an embedding model and a language model.
3. Create a knowledge base and vector database.
4. Accept a user question.
5. Retrieve relevant documents.
6. Combine the retrieved documents with the previous conversation.
7. Generate an answer using the language model.
8. Store the current question and answer in conversation history.
9. Repeat for subsequent questions.

CODE

Cell 1: Install libraries

```
!pip -q install langchain langchain-community langchain-huggingface faiss-cpu sentence-transformers transformers sentencepiece
```

Cell 2: Python program

```
import torch

from transformers import pipeline

from langchain_core.documents import Document

from langchain_community.vectorstores import FAISS

from langchain_huggingface import HuggingFaceEmbeddings


# Load embeddings

embeddings = HuggingFaceEmbeddings(

    model_name="sentence-transformers/all-MiniLM-L6-v2"

)
```

```
# Knowledge base
documents = [
    Document(page_content="Python is a programming language used in AI."),
    Document(page_content="Machine learning learns patterns from data."),
    Document(page_content="Deep learning uses multi-layer neural networks."),
    Document(page_content="Python libraries such as NumPy support numerical computing.")
]

# Create vector database
db = FAISS.from_documents(documents, embeddings)

# Load language model
generator = pipeline(
    "text2text-generation",
    model="google/flan-t5-small",
    device=0 if torch.cuda.is_available() else -1
)

# Conversation history
history = []

print("Context-Aware AI Chatbot")
print("Type 'exit' to stop.\n")

while True:
    question = input("You: ")

    if question.lower().strip() == "exit":
        print("Chatbot: Goodbye!")
        break
```

```
    # Retrieve relevant documents
    results = db.similarity_search(question, k=2)
    context = "\n".join(doc.page_content for doc in results)

    # Use recent conversation history
    previous = "\n".join(history[-4:])

    prompt = (
        "Answer using the knowledge context and conversation history.\n"
        "If the answer is unavailable, say you do not know.\n"
        f"Knowledge: {context}\n"
        f"Conversation: {previous}\n"
        f"Question: {question}\n"
        "Answer:"
    )

    answer = generator(
        prompt,
        max_new_tokens=100,
        do_sample=False
    )[0]["generated_text"]

    print("Chatbot:", answer, "\n")

    history.append("User: " + question)
    history.append("Assistant: " + answer)
```

INPUT

You: What is Python used for?

You: Which libraries help with numerical computing?

You: exit

OUTPUT

Context-Aware AI Chatbot

You: What is Python used for?

Chatbot: Python is a programming language used in AI.

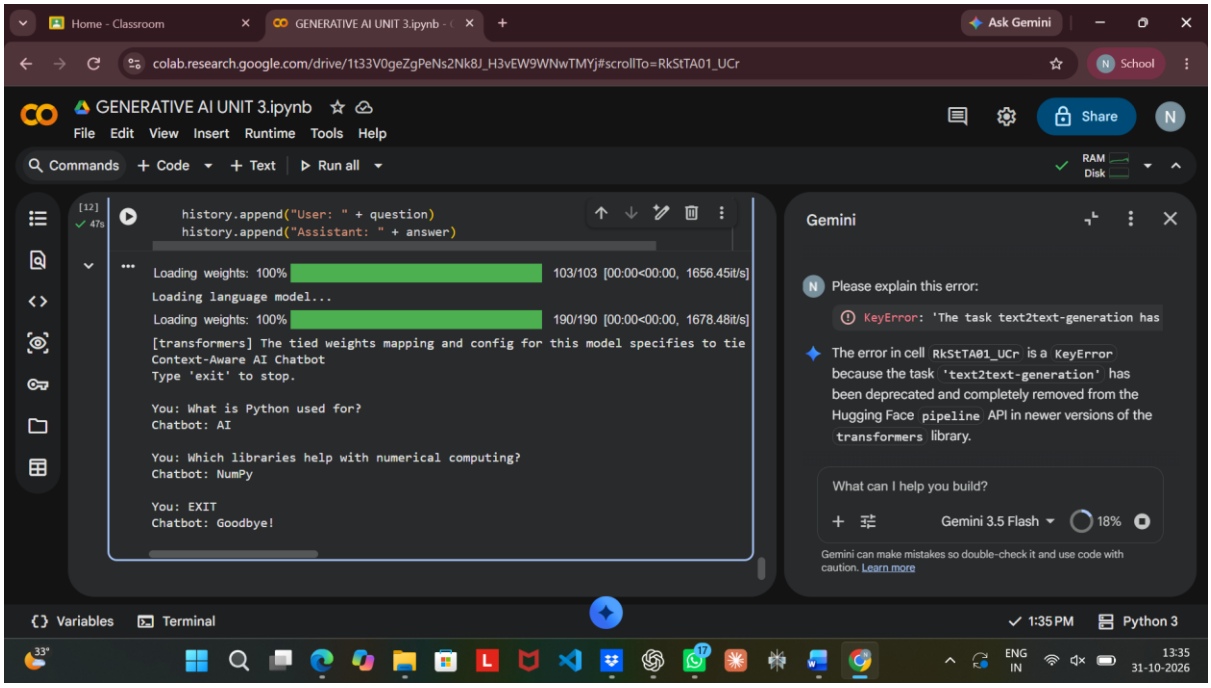
You: Which libraries help with numerical computing?

Chatbot: Python libraries such as NumPy support numerical computing.

You: exit

Chatbot: Goodbye!

The actual generated responses may vary.



RESULT

A context-aware chatbot was implemented using LangChain-compatible components, vector retrieval, an LLM and conversation history